

CLAUDE CODE DEEP DIVE

Tutorial1

Source: Notes.md

1. Claude Code is an Agentic Coding tool which is created by Anthropic. Other coding tools include Co-Pilot, Cursor, Winsurf etc.
2. First install the claude code on your system using `curl -fsSL https://claude.ai/install.sh | bash`. Now, Go to `Tutorial1` and type `claude` in vscode terminal. Before that take the subscription plan from <https://claude.com/pricing>
3. Now, start with chatting as `Can you provide me a summary of what this project is?`
4. First type `/terminal-setup` command for key bindings for new line, so when you are chatting and you have to go to new line then press `shift+enter` for new line in claude terminal.
5. type `can you switch to a new branch called claude-edits`, so claude will switch to a new branch because claude has knowledge about the git branches. It can run bash commands also. It can do stage and commit as well and resolve conflicts as well in git after merging. You can check changed branches in VS Code.
6. Put some github code in current local folder. Now, it's a good idea to run `\init` command initially for `claude.md`, so claude code scans entire codebase of current folder and summarize and store into [claude.md](#) file.
7. You can change the project structure in [claude.md](#), for example, you can add a folder as `hooks` and then claude will understand it and then for your project, you can type, `can you create a hook to store user's theme preference in, when they toggle the theme on site? Store the value in local storage for next time. Don't use the hook anywhere yet.`
8. You can add memory for [claude.md](#) directly from the chat session. For example, type `# When making new page components, always add a link to that page in the header`. `#` is to memorize or you can use `/memory` command. Select `Project Memory` and check the [claude.md](#) file now.
9. You can type `Can you add a new /about page with only an h2 title and a single line of lorem as content.`
10. To select a file in claude code terminal, use `@Notes.md` and then press `Tab` key. So, it will provide the context. For example, type `Convert @Notes.md to pdf` and press enter. You can add multiple files using `@` symbol in the prompt. If you select few lines of a file then it will be displayed below the claude terminal.

We can also add images as context in the claude code terminal using drag and drop of images in the claude code terminal.

11. You can use `/exit` command to exit the chat session completely. `/clear` command is used to clear the entire session context and chat history.

`/compact` command is used to compacts the chat and context into a small summary.

If you press ESC twice then it rewind to a previous point in the session.

12. If you have used `/exit` command then you can put `/resume` again after opening the claude terminal to see previous chat sessions and context and resume from those sessions/contexts.

13. Reference Material: <https://code.claude.com/docs/en/overview>

14. You can put `can you add a commit` for git commands.

15.

Planning and Thinking:

Claude Code can do planning and thinking. Planning and Thinking both are different things. Thinking is what claude code to do when you ask claude code to think about a solution before writing the code and Planning is to make a plan for how to add intensive features and ask you to approve it.

Type `Shift+Tab` to switch between plan mode and accept edit mode. Now, in the plan mode, you can write something as `Can you make a custom component for an Avatar (no pic, only initial) and find any places in the project where it can replace Avatar-like templates` and type Enter. You can put instructions in [claude.md](#) and put an attachment in claude code terminal.

Thinking mode is good when claude code works on more complex logic. It takes more tokens. For example, `Implement a comment system into the application, where users can authenticate and then comment on blog posts. Think hard about this implementation, including the database schema, authentication services, moderation and real-time updates.`. Here, in this example, “Think” will trigger the thinking mode and in claude code terminal, use planning mode also to know, it thinks. So, “Think” word in the prompt enables the thinking mode here. Also, Put task in chunks and then guide model for each small task.

In prompt, you can write `Think harder` or `ultra think` to think more but it will consume more tokens. Also, if after think, it gives a larger plan then accomplish the tasks in chunks or phases and don't put everything in one go.

16. MCP/MCP Servers:

MCP servers give ability to claude code to connect and communicate with external data sources, services and APIs.

MCP stands for model context protocols. It is designed by Anthropic in such a way that AI Models can interact with external data sources by providing external tools and contexts by MCP servers.

MCP servers have different tools to do different things and interact with different sources.

So, you can think MCP servers as to plugin claude code or any other client and claude is directed by an AI model to interact with the data sources for the specific tool.

For example, `Supabase MCP Server` have tools like `list_tables()`, `deploy_edge_function()`, `execute_sql()` etc. So, if I install `Supabase MCP Server` the Claude code will have access to all these tools.

Another MCP server is `Playwright MCP` which provides browser automation capability using Playwright. This server enables LLMs to interact with web pages through structured accessibility snapshots, bypassing the need for screenshots or visually-tuned models.

First run some commands on mac to add MCP servers then in Claude code terminal type `/mcp` for list of MCP servers.

17. Subagents:

Each subagent can be `isolated` and configured to work on particular tasks. We can make subagents who can work as `Unit Tester` or `Security Auditor` or `Code Reviewer` or `UX Reviewer` etc. Each of these sub-agents are independent to each other and have their own system prompts and tools and context window.

Claude Code can delegate tasks to subagents. Each subagent has specialized expertise which we can fine-tune in a specific area through detailed system prompt. Since each subagent has the isolated context window, it keeps context cleaned and focused leading to better results. It also reduces context overload in the main session.

In Claude code terminal, type `/agent` for list of agents. Now, create a new agent here on project level and describe what it will do.

For example, you can describe as `Expert UI and UX engineer who reviews the UI & UX of React components in a browser using Playwright, take screenshots and then offers feedback on how to improve the component in terms of visual design, user experience and accessibility.` and hit enter. Claude code will create this new agent.

You can see the newly created agent in agents folder inside `.claude` folder with the markdown file. You can edit this markdown file also if you want.

18. For github code and its review, you can use `/install-github-app` and then choose the current or different directory and then install and authorize Claude on github. Now, create a long-lived token. Now, create and merge pull request. Now, create an issue in github repo and put comment like `@claude can you fix it` the Claude will fix it.